IMY 220

Assignment 6

Docker Compose

Due: 21 October at 13:00

The submission instructions are available on ClickUP. Any deviation from these instructions will result in a 10% deduction from your mark.

Instructions

This is an individual, take-home assignment.

In this assignment, you will configure and containerise a supplied full-stack **PhotoShare application** using **Docker** and **Docker Compose**.

The supplied application contains:

- a React client;
- an Express API;
- a MongoDB database connection;
- the basic PhotoShare post functionality;
- the required project structure.

Your task is to configure the application so that the complete PhotoShare system runs as a Docker Compose stack containing three services:

- a React client;
- an Express API;
- a MongoDB database.

You will create and configure the required Dockerfiles and Docker Compose configuration.

Your completed stack must use:

- separate containers for the client, API and database;
- Docker Compose to manage all services;
- a Docker network for communication between services;
- a Docker volume for persistent MongoDB data;
- environment configuration;
- service dependencies;
- a MongoDB healthcheck;
- restart policies.

The complete application must be started and managed using Docker Compose.

Ensure that your student number is included as a comment at the top of all Dockerfiles and the `compose.yml` file. Replace `uXXXXXXX` with your student number.

Student Number: uXXXXXXX

Do not substantially modify the supplied React, Express or MongoDB application functionality.

Scenario

You are continuing development on the PhotoShare social media application.

PhotoShare consists of three parts:

React Client
↓
Express API
↓
MongoDB

During development, these parts can be started and managed separately. However, PhotoShare must now be configured as a **multi-container application**.

Docker Compose must be used to manage the complete application as a single stack. This reflects the purpose of Compose described in the lecture: multiple related containers can be defined and managed together rather than individually.

The React client must communicate with the Express API.

The Express API must communicate with MongoDB.

The application must not use `localhost` when one container communicates with another container.

MongoDB data must persist when the containers are stopped and started again.

When the completed stack is running, a user must be able to:

- open the PhotoShare React application;
- view existing posts;
- create a new post;
- retrieve post data through the Express API;
- store post data in MongoDB.

Supplied Project

Download and extract the supplied starter project.

Open the project folder in Visual Studio Code.

The project contains:

assignment-6/

```
├── client/
│   ├── src/
│   ├── package.json
│   ├── package-lock.json
│   └── ...
│
├── server/
│   ├── server.js
│   ├── package.json
│   ├── package-lock.json
│   └── .env
│
└── compose.yml
```

The `client` folder contains the supplied React application.

The `server` folder contains the supplied Express API and MongoDB connection functionality.

A basic `compose.yml` file has been supplied containing a MongoDB service. This service allows the supplied application to be tested before completing the Docker configuration.

Before starting the assignment, run:

```
docker compose up -d
```

Then start the supplied Express server and React client using their existing npm scripts.

Confirm that the supplied PhotoShare application loads existing posts and allows a new post to be created.

You must extend the supplied Docker Compose configuration and create the additional Docker configuration required to run the complete application as a multi-container stack.

Section 1: React Dockerfile

Create a file called `Dockerfile` inside the supplied `client/` folder.

The Dockerfile must:

- use an appropriate Node.js base image;
- set an appropriate working directory;
- copy the required package files;
- install the project dependencies;
- copy the React application files;
- expose the port used by the supplied React application;
- start the React application using the supplied npm script.

Create a `.dockerignore` file inside the `client` folder to prevent unnecessary files from being copied into the image.

Section 2: Express Dockerfile

Create a file called `Dockerfile` inside the supplied `server/` folder.

The Dockerfile must:

- use an appropriate Node.js base image;
- set an appropriate working directory;
- copy the required package files;
- install the project dependencies;
- copy the Express application files;
- expose port `3001`;
- start the Express application using the supplied npm script.

Create a `.dockerignore` file inside the `server` folder to prevent unnecessary files from being copied into the image.

Docker Compose does not replace the Dockerfiles used to construct the individual application images; Compose manages those containers as part of the larger stack.

Section 3: MongoDB Service

Open the supplied `compose.yml` file.

The supplied Compose file already contains a basic MongoDB service called:

`db`

The service uses the official MongoDB image and exposes MongoDB on port `27017`.

Extend the supplied db service by configuring an appropriate restart policy.

Create a named Docker volume called:

`mongo_data`

Attach the volume to the MongoDB service so that MongoDB stores its database files inside:

/data/db

The volume must be declared in the top-level `volumes` section of the Compose file.

Section 4: MongoDB Healthcheck

Add a `healthcheck` to the `db` service.

The healthcheck must verify that MongoDB is available.

Configure the healthcheck with an appropriate:

- test;
- interval;
- timeout;
- number of retries.

The MongoDB container must report a healthy status when the database is available.

Section 5: Express API Service

Create a second service called:

api

The service must build the Express API using the Dockerfile inside:

./server

Give the container the name:

photoshare_api

Expose the supplied Express API using:

3001:3001

Configure an appropriate restart policy.

The API must receive its MongoDB connection configuration using the supplied environment configuration.

The MongoDB connection must use the Compose service name:

db

as the database host.

Do not use:

localhost

to connect from the API container to MongoDB.

The API service must depend on the MongoDB service.

Configure the dependency so that the API starts after the MongoDB service is healthy.

Section 6: React Client Service

Create a third service called:

client

The service must build the React application using the Dockerfile inside:

./client

Give the container the name:

photoshare_client

Expose the React application using the port specified by the supplied project.

Configure an appropriate restart policy.

The supplied React application must remain able to communicate with the Express API through the exposed API port.

The client service must depend on the API service.

Section 7: Environment Configuration

Use the supplied `.env` file to store the required environment configuration.

Update the supplied MongoDB connection string so that the API container connects to MongoDB using the Docker Compose service name:

`db`

Configure the `api` service to retrieve the required environment values from the supplied `.env` file.

Do not hard-code the MongoDB connection string directly inside the Express application source code.

Section 8: Docker Network

Create a Docker network called:

photoshare_network

Declare the network in the top-level `networks` section of the Compose file.

Attach the:

- `client`;
- `api`;
- `db`;

services to the network.

Containers that communicate directly with other containers must use the appropriate Docker Compose service name.

Section 9: Build and Run the Stack

Open a terminal inside the main `assignment-6` folder.

Build and start the complete application using:

docker compose up -d --build

Docker Compose must successfully start all three services.

Use Docker Desktop to confirm that the following containers are running:

photoshare_client
photoshare_api
photoshare_db

Confirm that the MongoDB container reports a healthy status.

Open the React application in your browser.

The supplied PhotoShare posts must be retrieved through the Express API and displayed in the React application.

Section 10: Test the Full Application

Use the React interface to create a new PhotoShare post.

The request must flow through the complete application:

React Client
↓
Express API
↓
MongoDB
↓
Express API
↓
React Client

Confirm that the newly created post appears in the React application.

Stop and remove the containers using:

docker compose down

Start the application again using:

docker compose up -d

Open the React application again.

The newly created post must still be available.

This demonstrates that the MongoDB data is stored using the Docker volume rather than being lost when the containers are removed.

Section 11: Docker Compose CLI

Use Docker Compose to inspect the complete application.

Run:

docker compose logs

Confirm that logs are available for the services in the stack.

You may use:

docker compose logs -f

to follow the logs while the application is running.

Stop the services using:

docker compose stop

Start the services again using:

docker compose start

Confirm that the complete application becomes available again.

Expected Application Behaviour

When the assignment is complete, the application must run as the following Docker Compose stack:

PhotoShare Docker Compose Stack

├── client

```
│   └── React
│
├── api
│   └── Express
│
└── db
    ├── MongoDB
    └── mongo_data
```

All three services must be attached to:

photoshare_network

The complete application flow must be:

User
↓
React Client
↓
Express API
↓
MongoDB

The React application must be accessible from the browser.

The Express API must successfully communicate with MongoDB.

MongoDB must use a Docker volume for persistent storage.

The MongoDB service must report a healthy status.

Post data must remain available after the containers are stopped, removed and started again.

Testing Your Application

Before submitting, test the complete application.

Confirm that:

- the React Docker image builds successfully;
- the Express Docker image builds successfully;
- all three Compose services start successfully;
- the React application is accessible in the browser;
- the Express API is accessible;
- the API successfully connects to MongoDB;

- MongoDB reports a healthy status;
- all three services use the required Docker network;
- the API uses the `db` service name to communicate with MongoDB;
- environment configuration is correctly supplied to the containers;
- existing posts are displayed;
- a new post can be created;
- the new post is stored in MongoDB;
- the new post remains available after `docker compose down` and `docker compose up -d`;
- the Compose logs can be viewed;
- the complete application runs without container errors.

Submission Files

Submit the complete Assignment 6 project.

Do not include any `node_modules` folders.

Your ZIP file should contain:

assignment-6/

```
├── client/
│   ├── src/
│   ├── package.json
│   ├── package-lock.json
│   ├── Dockerfile
│   ├── .dockerignore
│   └── ...
│
├── server/
│   ├── server.js
│   ├── package.json
│   ├── package-lock.json
│   ├── Dockerfile
│   ├── .dockerignore
│   └── .env
│
└── compose.yml
```

Submission Instructions

Delete all `node_modules` folders before creating your ZIP file.

ZIP the complete project folder using a tool such as 7-Zip or WinRAR.

Name the ZIP file:

a6-{student_number}.zip

Replace `{student_number}` with your student number.

Submit your ZIP file before the deadline to avoid a late submission.